



Kimia

Documentation du code source

Application interne de gestion et site public pour une entreprise créative :
demandes, clients, projets, prestations, activités, planning, contenu, documents,
notifications, finances, sécurité.

Backend : FastAPI · SQLAlchemy 2 · PostgreSQL · Alembic · JWT · Argon2 · WebSocket

Frontend : HTML · CSS · JavaScript vanilla · PWA (Service Worker)

Version 0.1 — 28 septembre 2026

Sommaire

1. Vue d'ensemble	4
1.1 Principe fondamental	4
1.2 Architecture	4
1.3 Choix structurants	4
2. Structure du projet et mise en route	5
2.1 Arborescence	5
2.2 Démarrage local	5
2.3 Configuration (.env)	6
3. Backend	6
3.1 Configuration et base de données	6
3.2 Modèles SQLAlchemy	6
3.3 Schemas Pydantic	7
3.4 Authentification et sécurité des comptes	7
3.5 Permissions par rôle	8
3.6 Logique métier implémentée	8
Cycle de vie d'une demande	8
Création d'un projet et génération des activités	8
Affectation des activités	9
Planning et conflits	9
Calendrier éditorial	9
Finance (MVP)	9
Documents	10
3.7 Temps réel : WebSocket	10
3.8 Notifications	10
4. Frontend	11
4.1 Site public	11
4.2 Application interne : architecture	11
4.3 Le contrat d'une page	11
4.4 Comportement « sans rechargement »	11
4.5 Pages de l'application	12
4.6 PWA (application interne uniquement)	12
4.7 Jeton et sécurité côté navigateur	12

5. Tests	13
6. Hypothèses, limites et suite	13
6.1 Hypothèses retenues (points restés ouverts dans le cahier des charges)	13
6.2 Ce qui n'est pas (encore) fait	13
6.3 Avant la mise en production	14
Annexe A — Schéma de la base de données	15
Annexe B — Référence de l'API	22
Annexe C — Comment étendre Kimia	24
Ajouter une prestation avec son formulaire et ses activités (sans code)	24
Ajouter une nouvelle ressource API	24
Ajouter un écran à l'application interne	24

1. Vue d'ensemble

Kimia est composée de **deux interfaces** qui partagent une seule API REST :

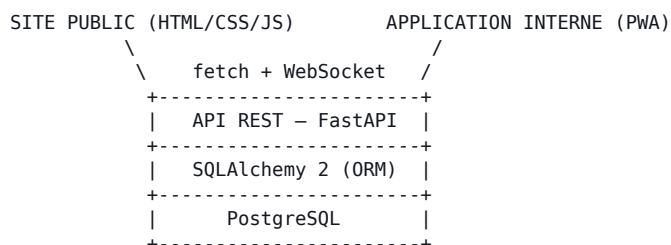
- **Site public** (frontend/public) : vitrine, catalogue de services, portfolio, contact et formulaire de demande de prestation. Ce n'est **pas** une PWA.
- **Application interne** (frontend/app) : PWA privée pour l'équipe (CEO, DA, CM) — authentification JWT, mise à jour en temps réel par WebSocket, pages par métier.

1.1 Principe fondamental

« Le code fournit le moteur. Les données fournissent la configuration. » Les catégories, prestations, types d'événements, champs de formulaire, modèles d'activités, rôles, permissions, compétences et plateformes sont des **lignes en base**, jamais des constantes dans le code. Ajouter une prestation ou un champ de formulaire ne demande aucun déploiement.

Les seules valeurs figées dans le code sont des **machines à états** métier fermées : statuts de demande, de projet, d'activité, de publication, priorités, formats de publication, statut d'actionnariat.

1.2 Architecture



Temps réel : écriture DB -> router -> manager.broadcast_sync()
 -> WebSocket -> frontend/services/websocket.js
 -> re-fetch ciblé -> DOM de la page concernée

1.3 Choix structurants

- **Routers synchrones + SQLAlchemy classique** : simple à lire et à tester. Le pont vers le WebSocket (asynchrone) est isolé dans `ws_manager.broadcast_sync()`.
- **Couche « services » uniquement quand nécessaire** : un seul fichier (`project_service.py`) pour la création d'un projet, qui touche plusieurs modèles. Pas de dossier `repositories/`.
- **Le backend est l'unique autorité** : chaque route vérifie rôle, propriété et transitions. Masquer un menu côté frontend n'est qu'un confort d'affichage.
- **Le frontend ne recharge jamais la page** pour synchroniser des données : aucun `location.reload()`. Le serveur reste la source de vérité.

2. Structure du projet et mise en route

2.1 Arborescence

```
kimia/
├── backend/
│   ├── app/
│   │   ├── main.py          # app FastAPI, CORS, lifespan, routers
│   │   ├── config.py        # variables d'environnement, seuils métier
│   │   ├── database.py      # engine, Session, Base, get_db()
│   │   ├── seeds.py         # données de configuration initiales
│   │   ├── create_user.py    # création du premier compte en ligne de commande
│   │   ├── models/          # tables SQLAlchemy (auth, business, workflow, content, finance)
│   │   ├── schemas/         # schemas Pydantic (entrées/sorties API)
│   │   ├── routers/         # HTTP : auth, users, clients, public, requests, projects,
│   │   │                       # activities, publications, documents, finance,
│   │   │                       # notifications, security, ws
│   │   ├── services/project_service.py # création projet + génération d'activités
│   │   └── core/            # security.py (Argon2/JWT), deps.py, ws_manager.py
│   ├── alembic/             # migrations
│   │   └── alembic.ini
│   ├── tests/               # pytest (API)
│   └── requirements.txt      .env.example
├── frontend/
│   ├── public/              # site vitrine (index, services, portfolio, about, contact, request)
│   └── app/                  # PWA interne
│       ├── login.html dashboard.html pages/*.html
│       ├── components/shell.js # coquille : garde d'accès, menu par rôle, temps réel
│       ├── services/api.js     websocket.js
│       ├── state/store.js      utils/ui.js
│       ├── assets/js/pages/*.js # une « page » = un module
│       └── sw.js manifest.json offline.html
```

2.2 Démarrage local

```
cd backend
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env # renseigner DATABASE_URL et SECRET_KEY

alembic revision --autogenerate -m "initial schema"
alembic upgrade head
python -m app.seeds # rôles, permissions, catégories, compétences...
python -m app.create_user --name "Vous" --email vous@exemple.com --role CEO --cofounder
uvicorn app.main:app --reload # API sur :8000, docs sur /docs

cd ../frontend && python -m http.server 5500
# site public : http://localhost:5500/public/
# application : http://localhost:5500/app/login.html
```

Premier utilisateur. Il n'existe volontairement aucune inscription publique. Le premier compte CEO se crée en ligne de commande : `python -m app.create_user --name ... --email ... --role CEO --cofounder` (mot de passe demandé de façon masquée, 8 caractères minimum). Les comptes suivants se créent depuis la page *Équipe* par le CEO.

2.3 Configuration (.env)

Variable	Défaut	Rôle
DATABASE_URL	postgresql+psycopg2://kimia:kimia@localhost/kimia	Connexion PostgreSQL
SECRET_KEY	CHANGE_ME_IN_PROD	Signature des JWT — à remplacer par une valeur longue et aléatoire
ACCESS_TOKEN_EXPIRE_MINUTES	1440	Durée de vie du JWT (24 h, pas de refresh token)
MAX_LOGIN_ATTEMPTS	5	Échecs avant blocage du compte (seuil à ajuster)
ACCOUNT_LOCK_MINUTES	14	Durée du blocage temporaire
MAX_DOCUMENT_SIZE_MB	20	Taille maximale d'un document
DEFAULT_CURRENCY	XOF	Devise par défaut des montants
CORS_ALLOWED_ORIGINS	localhost:5500, :8000	Origines autorisées (à restreindre en production)

Côté frontend, l'URL de l'API se règle dans `assets/js/config.js` (`window.KIMIA_API`), présent dans `public/` et `app/`.

3. Backend

3.1 Configuration et base de données

`config.py` ne contient que de la configuration (classe `Settings`, `pydantic-settings`). `database.py` ne contient que la connexion : `engine`, `SessionLocal`, la Base déclarative et la dépendance `get_db()`. Les deux responsabilités ne sont volontairement pas mélangées.

```
def get_db() -> Generator[Session, None, None]:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

3.2 Modèles SQLAlchemy

Les **32 tables** sont réparties en cinq fichiers, importés ensemble par `models/__init__.py` pour que la registry SQLAlchemy résolve toutes les relations (indispensable à `alembic --autogenerate`). Le détail complet colonne par colonne est en annexe A.

Fichier	Tables
<code>auth.py</code>	<code>roles</code> , <code>permissions</code> , <code>role_permissions</code> , <code>skills</code> , <code>users</code> , <code>user_skills</code> , <code>security_events</code> , <code>audit_logs</code>
<code>business.py</code>	<code>categories</code> , <code>services</code> , <code>event_types</code> , <code>service_event_types</code> , <code>form_fields</code> , <code>service_form_fields</code> , <code>clients</code>
<code>workflow.py</code>	<code>requests</code> , <code>request_status_history</code> , <code>projects</code> , <code>project_services</code> , <code>activity_templates</code> , <code>checklist_template_items</code> , <code>activities</code> , <code>activity_assignments</code> , <code>activity_checklists</code> , <code>documents</code> , <code>notifications</code>
<code>content.py</code>	<code>platforms</code> , <code>publications</code> , <code>publication_platforms</code>
<code>finance.py</code>	<code>revenues</code> , <code>payments</code> , <code>remunerations</code>

Décisions de modélisation qui comptent au quotidien :

- **Historisation.** Le prix d'une prestation vendue est copié dans `project_services.agreed_amount` : changer le catalogue ne modifie pas les anciens projets. Les activités et checklists d'un projet sont des *copies* des modèles ; `activities.template_id` reste nullable et ne sert qu'à la traçabilité.
- **Suppressions.** Les tables de catalogue (`categories`, `services`) sont protégées par `ON DELETE RESTRICT` et portent `is_active` : on désactive, on ne supprime pas. Les liens de jonction et les

lignes « filles » (checklists, historique) suivent le parent en CASCADE.

- **Demande ≠ projet.** `projects.request_id` est nullable et UNIQUE (relation 0..1 — 0..1) ; un projet interne n'a pas de demande. `client_id` est aussi nullable pour les projets internes.
- **Rôle ≠ compétence ≠ actionnariat.** Le rôle est une ligne de rôles, les compétences passent par `user_skills`, et `ownership_status` (COFOUNDER / COLLABORATOR) est une colonne à part, sans lien avec la rémunération.
- **Formulaires dynamiques.** Les réponses d'une demande sont stockées dans `requests.form_data` (JSONB) — pas une colonne par champ possible.
- **Finance.** Un *revenu* (montant dû) reçoit des *paiements* partiels ; les *rémunérations* sont rattachées à la prestation du projet. Aucun booléen « payé ».

Extrait — la table `projects` (lien optionnel avec la demande) :

```
class Project(Base):
    __tablename__ = "projects"
    id: Mapped[int] = mapped_column(primary_key=True)
    client_id: Mapped[int | None] = mapped_column(
        ForeignKey("clients.id", ondelete="SET NULL"), nullable=True)
    request_id: Mapped[int | None] = mapped_column(
        ForeignKey("requests.id", ondelete="SET NULL"), unique=True, nullable=True)
    project_type: Mapped[ProjectType] = mapped_column(nullable=False) # CLIENT | INTERNE
    category: Mapped[ProjectCategory] = mapped_column(nullable=False)
    status: Mapped[ProjectStatus] = mapped_column(default=ProjectStatus.A_PREPARER)
    ...
```

3.3 Schemas Pydantic

Les schemas ne sont **pas** les tables : `User` (SQLAlchemy), `UserCreate` (entrée) et `UserRead` (sortie) sont trois objets distincts. `UserRead` n'expose jamais `password_hash` — un test le vérifie. Les schemas de lecture utilisent `from_attributes=True` ; ceux de création n'acceptent que les champs que le client a le droit de fournir (par exemple, un client public ne peut pas choisir le statut d'une demande).

3.4 Authentification et sécurité des comptes

```
email + mot de passe -> Argon2 (verify) -> JWT {sub, role, iat, exp} -> Authorization: Bearer
<jeton>
```

- Pas d'inscription publique : aucune route ne crée un utilisateur hors CEO.
- JWT valable 24 h, sans refresh token. À l'expiration : **401** → le frontend efface la session et renvoie vers `login.html`.
- **Un JWT valide ne suffit pas.** `get_current_user` recharge l'utilisateur en base à chaque requête et refuse un compte désactivé (403).
- **Blocage temporaire.** Après 5 mauvais mots de passe, `locked_until` est posé à maintenant + 14 min. Pendant ce délai, même le bon mot de passe est refusé. Le CEO peut débloquent (POST `/api/auth/unlock/{id}`).
- **Journal de sécurité.** Chaque tentative écrit une ligne dans `security_events` (utilisateur si connu, email saisi, IP, type, succès). La page *Sécurité* du CEO les affiche, avec les comptes actuellement bloqués.
- Le message d'échec est identique pour « email inconnu » et « mauvais mot de passe » (pas d'énumération de comptes).

Les gardes de rôle sont des dépendances FastAPI :

```
def require_roles(*allowed_role_names: str):
    """Usage : Depends(require_roles("CEO", "DA"))"""

    def checker(current_user: User = Depends(get_current_user)) -> User:
        if current_user.role.name not in allowed_role_names:
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail="Accès non autorisé pour ce rôle",
            )
        return current_user

    return checker
```

3.5 Permissions par rôle

Le contrôle a lieu dans les routers, à deux niveaux : le **rôle** (`require_roles`) et, pour le CM, la **propriété** (« seulement ce qui m'est assigné »).

Domaine	CEO	DA	CM
Demandes, clients	Tout	Tout	Aucun accès
Projets	Tout	Tout	Ses projets uniquement (assigné à au moins une activité)
Activités	Tout + assigner	Tout + assigner	Les siennes ; modifie uniquement celles qui lui sont assignées
Publications	Valide / demande modif.	Valide / demande modif.	Crée, modifie, soumet (BROUILLON, À VALIDER)
Documents	Tout	Tout + suppression	Lecture/ajout, pas de suppression
Équipe / utilisateurs	Gestion complète	Vue équipe + compétences	Aucun accès
Finances	Tout	Aucun accès	Aucun accès
Sécurité	Tout	Aucun accès	Aucun accès

Les rôles sont testés par leur *nom* (« CEO », « DA », « CM »). Les tables permissions et `role_permissions` existent et sont alimentées par `seeds.py`, mais l'application des routes s'appuie pour l'instant sur les noms de rôle : voir la section « Limites et suite ».

3.6 Logique métier implémentée

Cycle de vie d'une demande

```
NOUVELLE -> A_CONTACTER -> EN_DISCUSSION -> CONFIRMEE -> (projet)
\_____ \_____ \-> REFUSEE | ANNULEE (CONFIRMEE -> ANNULEE possible)
```

Les transitions autorisées sont définies dans `routers/requests.py` (`ALLOWED_TRANSITIONS`) ; toute autre transition renvoie 400. Chaque changement écrit une ligne dans `request_status_history` (ancien statut, nouveau, auteur, motif). Le site public crée la demande : le client est retrouvé par email puis téléphone, sinon créé.

Création d'un projet et génération des activités

C'est la seule vraie couche « service » : elle manipule `Project`, `ProjectService`, `Activity` et `ActivityChecklist` dans une seule transaction.

```

def _generate_activities_from_templates(
    db: Session, project: Project, project_service: ProjectService
) -> None:
    templates = (
        db.query(ActivityTemplate)
        .filter(
            ActivityTemplate.service_id == project_service.service_id,
            ActivityTemplate.is_active.is_(True),
        )
        .order_by(ActivityTemplate.display_order)
        .all()
    )

    for template in templates:
        activity = Activity(
            project_id=project.id,
            project_service_id=project_service.id,
            template_id=template.id,
            required_skill_id=template.required_skill_id,
            name=template.name,
            description=template.description,
            priority=template.priority or ActivityPriority.NORMALE,
        )
        db.add(activity)
        db.flush()

        checklist_items = (
            db.query(ChecklistTemplateItem)
            .filter(ChecklistTemplateItem.activity_template_id == template.id)
            .order_by(ChecklistTemplateItem.display_order)
            .all()
        )
        for item in checklist_items:
            db.add(
                ActivityChecklist(
                    activity_id=activity.id,
                    label=item.label,
                    display_order=item.display_order,
                )
            )

```

Modifier un modèle d'activité plus tard n'affecte donc pas les projets existants. Une activité peut aussi être créée à la main (POST /api/activities) : `template_id` reste alors vide.

Affectation des activités

GET /activities/{id}/suggested-users renvoie les utilisateurs ayant la compétence requise ; l'interface les marque d'une étoile. **Rien n'est affecté automatiquement** : le CEO ou le DA valide via POST .../assignments, ce qui crée une notification pour la personne concernée.

Planning et conflits

Il n'existe **pas de table planning** : le planning est une vue des activités (`start_at/end_at`). Le filtre `?mine=true` donne le planning personnel. La détection de chevauchement est faite dans `pages/planning.js` : elle produit un avertissement visible, jamais un blocage.

Calendrier éditorial

Une publication est distincte d'une activité. Le CM peut poser BROUILLON ou À VALIDER ; seuls CEO/DA posent VALIDÉ ou À MODIFIER (le backend renvoie 403 sinon). Les plateformes sont une relation N-N via `publication_platforms`.

Finance (MVP)

```

ProjectService --> Revenue --> Payment (N paiements partiels)
ProjectService --> Remuneration --> User
solde : restant = agreed_amount - somme(paiements) ; reste Kimia = encaissé - rémunérations

```

Tout est réservé au CEO. GET /finance/project-services/{id}/detail renvoie le solde et les lignes détaillées. Pas de comptabilité, fiscalité, dividendes ou trésorerie : hors périmètre MVP.

Documents

Les fichiers sont écrits sur disque (backend/storage/documents, nom préfixé par un UUID) ; PostgreSQL ne garde que les métadonnées. La taille est contrôlée côté serveur (413 au-delà de la limite) et côté navigateur.

3.7 Temps réel : WebSocket

Un canal `/ws?token=<JWT>` par utilisateur connecté. Le jeton invalide est refusé par une fermeture de code **4401** (le client sait alors qu'il ne doit pas reboucler). Les routers synchrones publient par `manager.broadcast_sync(...)` ; la boucle `asyncio` est capturée au démarrage (lifespan dans `main.py`).

```
def broadcast_sync(self, event: dict, exclude_user_id: int | None = None) -> None:
    """À appeler depuis un endpoint/route sync après un commit DB."""
    if self.loop is None:
        return
    asyncio.run_coroutine_threadsafe(self.broadcast(event, exclude_user_id), self.loop)
```

Format d'événement : `{"type": "request.created", "data": {"id": 42}}`. Le message ne contient **que des identifiants** : le frontend recharge lui-même la donnée via l'API (donc soumise aux permissions). Aucune donnée sensible ne transite par le WebSocket.

Événement	Émis quand	Destinataires
<code>request.created</code>	Un visiteur envoie une demande	Tous les connectés (le frontend filtre par rôle)
<code>request.updated</code>	Changement de statut d'une demande	Tous
<code>project.created</code> / <code>project.updated</code>	Création / modification d'un projet	Tous
<code>activity.created</code> / <code>updated</code> / <code>completed</code>	Création, modification, passage à TERMINÉE, case de checklist	Tous
<code>activity.assigned</code>	Affectation d'une personne	Tous
<code>publication.created</code> / <code>updated</code> / <code>validated</code>	Cycle du calendrier éditorial	Tous
<code>document.created</code>	Ajout d'un document	Tous
<code>notification.created</code>	Une notification est créée pour vous	La personne concernée uniquement

À savoir. Les événements généraux sont diffusés à tous les utilisateurs connectés : le message ne contient qu'un identifiant et chaque écran re-télécharge ses données avec les droits de l'utilisateur, donc rien n'est exposé. Cela reste un point à resserrer (diffusion par rôle ou par projet) si l'équipe grandit.

3.8 Notifications

Une notification est une ligne de notifications (utilisateur, type, titre, message, projet/activité liés, lu ou non) et un événement WebSocket `notification.created`. Aujourd'hui, l'affectation d'une activité en crée une. Les autres déclencheurs du cahier des charges (nouvelle demande pour CEO/DA, échéance proche, publication à valider...) suivent le même schéma : voir « Limites et suite ».

4. Frontend

4.1 Site public

Six pages statiques générées avec le même gabarit (en-tête, navigation, pied de page) et une feuille de style commune `public.css`. Le design suit l'univers de l'imprimerie : les trois encres (magenta, cyan, jaune) et une cible de repérage. Les préférences de thème sombre et de réduction des animations sont respectées.

- **services.html** : catalogue lu depuis `GET /api/public/services`, groupé par catégorie.
- **request.html** : le **formulaire est construit dynamiquement** depuis la configuration (champs communs + `form_fields` de la prestation choisie + type d'événement si applicable). Les réponses spécifiques partent dans `form_data`. Un bouton désactivé pendant l'envoi évite les doubles soumissions.
- **portfolio.html** lit `assets/portfolio.json` (vide au départ : à remplir avec les réalisations).
- Tout texte venant de l'API est échappé avant insertion dans le DOM.

4.2 Application interne : architecture

```
API (services/api.js) -> State (state/store.js) -> UI (pages/*.js)
      ^               |
      +----- WebSocket (services/websocket.js) -+ événement => re-fetch ciblé
```

Fichier	Responsabilité
<code>services/api.js</code>	Client API centralisé (aucune URL d'API ailleurs). Ajoute le jeton Bearer ; sur 401 : efface la session et redirige vers le login.
<code>services/websocket.js</code>	Connexion, reconnexion à délai exponentiel (1 s → 30 s), ping, événement interne « resync » après reconnexion, polling de secours toutes les 90 s (onglet visible seulement).
<code>state/store.js</code>	Petit store observable : utilisateur, état réseau, état WebSocket, nombre de notifications non lues.
<code>components/shell.js</code>	Coquille commune : garde d'accès, menu filtré par rôle, bandeau hors connexion, pastille de synchro, cloche, boucle de rendu/rafraîchissement.
<code>utils/ui.js</code>	Composants réutilisables : échappement HTML, badges, toasts, états (chargement/vidé/erreur/hors ligne), modales et formulaires.
<code>assets/js/pages/*.js</code>	Une page = un module qui déclare titre, rôles autorisés, événements à écouter, <code>fetch()</code> , <code>render()</code> , <code>bind()</code> .

4.3 Le contrat d'une page

Ajouter un écran revient à écrire un module qui exporte cet objet ; la coquille s'occupe du reste (chargement, erreurs, rafraîchissement temps réel) :

```
export default {
  title: "Clients",
  roles: ["CEO", "DA"], // affichage refusé sinon (le backend refuse aussi)
  events: ["request.created"], // événements WebSocket qui déclenchent un rafraîchissement
  fetch: (ctx) => api.getClients(), // récupère les données
  render: (data, ctx) => `<table>...</table>`, // retourne du HTML (données échappées)
  bind: (view, data, ctx) => { ... }, // branche les boutons ; ctx.reload() rafraîchit
};
```

4.4 Comportement « sans rechargement »

- Après une action (créer, modifier, assigner, valider...) : bouton désactivé → appel API → **la réponse du serveur fait foi** → re-fetch de la page → mise à jour du DOM, avec conservation du défilement.
- Un événement WebSocket déclenche un rafraîchissement *silencieux* (regroupé sur 250 ms) : les lignes nouvelles ou modifiées sont brièvement surlignées.

- Si deux personnes modifient la même donnée, le dernier état renvoyé par le serveur s'affiche ; aucune modification locale n'est considérée comme définitive avant confirmation de l'API.
- Hors connexion : la pastille passe au rouge, un bandeau s'affiche ; au retour du réseau, la socket se reconnecte et l'écran se resynchronise.

4.5 Pages de l'application

Page	Rôles	Ce qu'elle fait
Tableau de bord	Tous	Indicateurs adaptés au rôle, activités en retard, échéances à 7 jours.
Demandes	CEO, DA	Liste filtrable, boutons de transition de statut (motif pour refus/annulation), détails + historique, « Créer le projet ».
Clients	CEO, DA	Liste, création, modification.
Projets	Tous (CM : les siens)	Liste, création multi-prestations, changement de statut.
Activités	Tous (CM : les siennes)	Filtre, statut, checklist, assignation avec suggestions par compétence.
Planning	Tous	Vue mensuelle groupée par jour, planning global ou personnel, alerte de chevauchement.
Publications	Tous	Calendrier éditorial, plateformes, soumission / validation.
Équipe	CEO, DA	Membres, compétences ; CEO : création, (dés)activation, déblocage.
Documents	Tous	Par projet : ajout (limite 20 Mo), téléchargement, suppression (CEO/DA).
Finances	CEO	Par prestation : dû, encaissé, restant, rémunérations ; saisie de revenus, paiements partiels, rémunérations.
Notifications	Tous	Liste, marquer comme lue, badge  en temps réel.
Sécurité	CEO	Événements de connexion, comptes bloqués, déblocage.

4.6 PWA (application interne uniquement)

- **manifest.json** : installable, affichage *standalone*, icônes 192/512 et « maskable ».
- **sw.js** : précache des ressources statiques essentielles, stratégie « réseau d'abord » avec repli cache pour les fichiers statiques, `page offline.html` pour les navigations sans réseau.
- **Les données métier ne sont jamais mises en cache** par le Service Worker (les appels API sont ignorés) : hors connexion, on n'affiche pas de données sensibles périmées.
- Le gestionnaire d'événement push affiche une notification système ; l'envoi côté serveur n'est pas encore implémenté (voir la section suivante).

4.7 Jeton et sécurité côté navigateur

Le JWT est conservé dans `localStorage` (l'API est sur une autre origine, donc pas de cookie de session). Le mot de passe n'est jamais stocké. Compromis assumé : tout contenu dynamique est échappé (`esc()`) pour limiter le risque XSS ; une politique CSP stricte est recommandée en production (section suivante).

5. Tests

Backend — 3 fichiers pytest (backend/tests), exécutés sur SQLite en mémoire (les types INET/JSONB de PostgreSQL y sont émulés dans `conftest.py`) : `DATABASE_URL=sqlite://` `pytest -q`. Ils couvrent :

- connexion, absence de `password_hash` dans les réponses, blocage après échecs répétés ;
- refus 401 sans jeton, 403 du CM sur clients/finances/équipe/sécurité ;
- parcours complet : demande publique → transitions (dont une transition refusée) → historique → projet → activités et checklists générées depuis le template ;
- finance : deux paiements partiels, solde restant, rémunérations ;
- le CM ne modifie que les activités qui lui sont assignées, et reçoit la notification ;
- publications : le CM ne peut pas valider, le CEO oui ; catalogue public avec champs de formulaire dynamiques.

Bout en bout (navigateur réel, Playwright) — vérifié pendant le développement, sur une API réelle et une base seedée : formulaire public dynamique → demande visible chez le CEO *sans rechargement* via WebSocket → confirmation → création du projet → activités générées → paiements partiels → assignation avec notification et badge  en temps réel chez le CM → menus filtrés par rôle → bandeau hors connexion → session expirée renvoyée au login. Ces scénarios ne sont pas livrés comme suite automatisée : voir « Limites et suite ».

Ce qui n'a pas été exécuté ici : les tests tournent sur SQLite, pas sur PostgreSQL. La compilation du DDL PostgreSQL des 32 tables a été validée, mais la première migration Alembic et un test sur un vrai PostgreSQL restent à faire dans votre environnement.

6. Hypothèses, limites et suite

6.1 Hypothèses retenues (points restés ouverts dans le cahier des charges)

Sujet	Choix retenu	Où le changer
Identifiants	Entiers auto-incrémentés (pas UUID)	Modèles + migration ; à trancher avant la première migration
Statut d'une prestation de projet	PLANIFIE / EN_COURS / TERMINE / ANNULE (non défini par le brief)	ProjectServiceStatus dans <code>models/workflow.py</code>
Devise	Mono-devise, XOF par défaut	DEFAULT_CURRENCY (.env) ; colonnes currency conservées
Suppression du catalogue	RESTRICT + <code>is_active</code>	ondelete des FK vers services/categories
Demande → projet	Au plus un projet par demande (UNIQUE)	Retirer unique sur <code>projects.request_id</code> si une demande peut en générer plusieurs
Taille max document	20 Mo (borne haute de 10–20 Mo)	MAX_DOCUMENT_SIZE_MB
<code>projects.category</code>	Obligatoire	nullable=True si un projet interne n'a pas de catégorie
Seuil de blocage	5 échecs, 14 minutes	MAX_LOGIN_ATTEMPTS / ACCOUNT_LOCK_MINUTES
Statut « SOUMIS » vs « À VALIDER »	À VALIDER (version la plus récente du brief)	PublicationStatus dans <code>models/content.py</code>

6.2 Ce qui n'est pas (encore) fait

- **Web Push côté serveur** : le Service Worker sait afficher une notification push, mais il manque l'abonnement du navigateur (clés VAPID), une table d'abonnements et l'envoi depuis le backend. Aujourd'hui les notifications sont temps réel *dans l'application ouverte* uniquement.

- **Notifications automatiques restantes** : nouvelle demande → CEO/DA, activité modifiée, échéance proche, publication à préparer / à valider, projet créé. Seule « activité assignée » crée une ligne notifications. Les échéances demandent une tâche planifiée.
- **Permissions pilotées par les données** : les routes testent le nom du rôle ; permissions / role_permissions ne sont pas encore consultées. Étape suivante logique : une dépendance `require_permission("finance.view", scope=...)`.
- **Journal d'audit** : la table `audit_logs` existe mais aucune route n'y écrit encore (ex. changement de dates d'un projet, qui/quand).
- **Édition du catalogue** : catégories, prestations, champs de formulaire et modèles d'activités se gèrent aujourd'hui en base / par seed ; il n'y a pas encore d'écran d'administration.
- **Migration initiale Alembic** : la configuration est prête, mais le fichier de migration n'est pas versionné (il se génère avec `alembic revision --autogenerate` sur votre PostgreSQL).
- **Diffusion WebSocket ciblée** et **pagination** des listes (les listes renvoient tout : acceptable pour une petite équipe).
- **Suite e2e automatisée** : les scénarios navigateur ont été joués manuellement par script pendant le développement ; à intégrer à la CI.

6.3 Avant la mise en production

- Remplacer `SECRET_KEY`, restreindre `CORS_ALLOWED_ORIGINS`, servir en HTTPS (obligatoire pour la PWA, le Service Worker et le Web Push).
- Ajouter une **limitation de débit** sur `/api/auth/login` et `/api/public/requests` (protège du bourrage d'identifiants et du spam de demandes) — le blocage de compte protège un compte, pas l'IP.
- Poser une **CSP** stricte sur l'application interne ; sauvegarder `backend/storage/documents` avec la base.
- Renseigner le portfolio, le numéro WhatsApp et l'email de la page Contact (valeurs d'exemple pour l'instant).

Annexe A — Schéma de la base de données

Généré automatiquement depuis les modèles SQLAlchemy (32 tables). Types en dialecte PostgreSQL ; « → » indique une clé étrangère et son comportement à la suppression.

categories

Colonne	Type	Contraintes	Référence
id	INTEGER	PK	
name	VARCHAR(100)	UNIQUE NOT NULL	
is_active	BOOLEAN	NOT NULL	

clients

Colonne	Type	Contraintes	Référence
id	INTEGER	PK	
name	VARCHAR(150)	NOT NULL	
phone	VARCHAR(30)		
email	VARCHAR(255)		
address	VARCHAR		
notes	VARCHAR		
created_at	TIMESTAMP WITH TIME ZONE	NOT NULL	
updated_at	TIMESTAMP WITH TIME ZONE	NOT NULL	

event_types

Colonne	Type	Contraintes	Référence
id	INTEGER	PK	
name	VARCHAR(100)	UNIQUE NOT NULL	

form_fields

Colonne	Type	Contraintes	Référence
id	INTEGER	PK	
label	VARCHAR(150)	NOT NULL	
field_type	VARCHAR(30)	NOT NULL	
options	JSONB		

permissions

Colonne	Type	Contraintes	Référence
id	INTEGER	PK	
code	VARCHAR(100)	UNIQUE NOT NULL	
description	VARCHAR		

platforms

Colonne	Type	Contraintes	Référence
id	INTEGER	PK	
name	VARCHAR(50)	UNIQUE NOT NULL	

roles

Colonne	Type	Contraintes	Référence
id	INTEGER	PK	
name	VARCHAR(50)	UNIQUE NOT NULL	
description	VARCHAR		
created_at	TIMESTAMP WITH TIME ZONE	NOT NULL	

skills

Colonne	Type	Contraintes	Référence
id	INTEGER	PK	
name	VARCHAR(100)	UNIQUE NOT NULL	

role_permissions

Colonne	Type	Contraintes	Référence
role_id	INTEGER	PK	→ roles (CASCADE)
permission_id	INTEGER	PK	→ permissions (CASCADE)

services

Colonne	Type	Contraintes	Référence
id	INTEGER	PK	
category_id	INTEGER	NOT NULL	→ categories (RESTRICT)
name	VARCHAR(150)	NOT NULL	
description	VARCHAR		
base_price	NUMERIC(12, 2)		
is_active	BOOLEAN	NOT NULL	

users

Colonne	Type	Contraintes	Référence
id	INTEGER	PK	
name	VARCHAR(150)	NOT NULL	
email	VARCHAR(255)	UNIQUE NOT NULL	
password_hash	VARCHAR	NOT NULL	
role_id	INTEGER	NOT NULL	→ roles (RESTRICT)
ownership_status	ownershipstatus	NOT NULL	
is_active	BOOLEAN	NOT NULL	
locked_until	TIMESTAMP WITH TIME ZONE		
failed_login_attempts	INTEGER	NOT NULL	
created_at	TIMESTAMP WITH TIME ZONE	NOT NULL	
updated_at	TIMESTAMP WITH TIME ZONE	NOT NULL	

activity_templates

Colonne	Type	Contraintes	Référence
id	INTEGER	PK	
service_id	INTEGER	NOT NULL	→ services (RESTRICT)
name	VARCHAR(200)	NOT NULL	
description	TEXT		
priority	activitypriority		
required_skill_id	INTEGER		→ skills (SET NULL)
display_order	INTEGER	NOT NULL	
is_active	BOOLEAN	NOT NULL	

audit_logs

Colonne	Type	Contraintes	Référence
id	INTEGER	PK	
user_id	INTEGER		→ users (SET NULL)
action	VARCHAR(100)	NOT NULL	
entity_type	VARCHAR(100)	NOT NULL	
entity_id	INTEGER	NOT NULL	
old_data	JSONB		
new_data	JSONB		
created_at	TIMESTAMP WITH TIME ZONE	NOT NULL	

requests

Colonne	Type	Contraintes	Référence
id	INTEGER	PK	
client_id	INTEGER	NOT NULL	→ clients (RESTRICT)
service_id	INTEGER	NOT NULL	→ services (RESTRICT)
event_type_id	INTEGER		→ event_types (SET NULL)
status	requeststatus	NOT NULL	
form_data	JSONB	NOT NULL	
notes	TEXT		
created_at	TIMESTAMP WITH TIME ZONE	NOT NULL	
updated_at	TIMESTAMP WITH TIME ZONE	NOT NULL	

security_events

Colonne	Type	Contraintes	Référence
id	INTEGER	PK	
user_id	INTEGER		→ users (SET NULL)
email	VARCHAR(255)		
ip_address	INET		
event_type	VARCHAR(50)	NOT NULL	
success	BOOLEAN	NOT NULL	
created_at	TIMESTAMP WITH TIME ZONE	NOT NULL	
event_metadata	JSONB		

service_event_types

Colonne	Type	Contraintes	Référence
service_id	INTEGER	PK	→ services (CASCADE)
event_type_id	INTEGER	PK	→ event_types (CASCADE)

service_form_fields

Colonne	Type	Contraintes	Référence
service_id	INTEGER	PK	→ services (CASCADE)
form_field_id	INTEGER	PK	→ form_fields (CASCADE)
display_order	INTEGER	NOT NULL	
is_required	BOOLEAN	NOT NULL	

user_skills

Colonne	Type	Contraintes	Référence
user_id	INTEGER	PK	→ users (CASCADE)
skill_id	INTEGER	PK	→ skills (CASCADE)

checklist_template_items

Colonne	Type	Contraintes	Référence
id	INTEGER	PK	
activity_template_id	INTEGER	NOT NULL	→ activity_templates (CASCADE)
label	VARCHAR(255)	NOT NULL	
display_order	INTEGER	NOT NULL	

projects

Colonne	Type	Contraintes	Référence
id	INTEGER	PK	
client_id	INTEGER		→ clients (SET NULL)
request_id	INTEGER	UNIQUE	→ requests (SET NULL)
name	VARCHAR(200)	NOT NULL	
description	TEXT		
project_type	projecttype	NOT NULL	
category	projectcategory	NOT NULL	
status	projectstatus	NOT NULL	
responsible_id	INTEGER		→ users (SET NULL)
start_date	DATE		
planned_end_date	DATE		
actual_end_date	DATE		
event_date	DATE		
event_time	TIME WITHOUT TIME ZONE		
event_location	VARCHAR(255)		
created_at	TIMESTAMP WITH TIME ZONE	NOT NULL	
updated_at	TIMESTAMP WITH TIME ZONE	NOT NULL	

request_status_history

Colonne	Type	Contraintes	Référence
id	INTEGER	PK	
request_id	INTEGER	NOT NULL	→ requests (CASCADE)
old_status	requeststatus		
new_status	requeststatus	NOT NULL	
changed_by	INTEGER		→ users (SET NULL)
reason	TEXT		
created_at	TIMESTAMP WITH TIME ZONE	NOT NULL	

documents

Colonne	Type	Contraintes	Référence
id	INTEGER	PK	
project_id	INTEGER	NOT NULL	→ projects (CASCADE)
uploaded_by	INTEGER		→ users (SET NULL)
name	VARCHAR(255)	NOT NULL	
path	VARCHAR(500)	NOT NULL	
mime_type	VARCHAR(150)	NOT NULL	
size	INTEGER	NOT NULL	
created_at	TIMESTAMP WITH TIME ZONE	NOT NULL	

project_services

Colonne	Type	Contraintes	Référence
id	INTEGER	PK	
project_id	INTEGER	NOT NULL	→ projects (CASCADE)
service_id	INTEGER	NOT NULL	→ services (RESTRICT)
agreed_amount	NUMERIC(12, 2)	NOT NULL	
currency	VARCHAR(3)	NOT NULL	
status	projectservicestatus	NOT NULL	
created_at	TIMESTAMP WITH TIME ZONE	NOT NULL	

publications

Colonne	Type	Contraintes	Référence
id	INTEGER	PK	
project_id	INTEGER		→ projects (SET NULL)
title	VARCHAR(200)	NOT NULL	
content	TEXT		
format	publicationformat		
scheduled_at	TIMESTAMP WITH TIME ZONE		
status	publicationstatus	NOT NULL	
media	JSONB		
notes	TEXT		
created_by	INTEGER		→ users (SET NULL)
created_at	TIMESTAMP WITH TIME ZONE	NOT NULL	
updated_at	TIMESTAMP WITH TIME ZONE	NOT NULL	

activities

Colonne	Type	Contraintes	Référence
id	INTEGER	PK	
project_id	INTEGER	NOT NULL	→ projects (CASCADE)
project_service_id	INTEGER		→ project_services (SET NULL)
template_id	INTEGER		→ activity_templates (SET NULL)
required_skill_id	INTEGER		→ skills (SET NULL)
name	VARCHAR(200)	NOT NULL	
description	TEXT		
status	activitystatus	NOT NULL	
priority	activitypriority	NOT NULL	
start_at	TIMESTAMP WITH TIME ZONE		
end_at	TIMESTAMP WITH TIME ZONE		
created_at	TIMESTAMP WITH TIME ZONE	NOT NULL	
updated_at	TIMESTAMP WITH TIME ZONE	NOT NULL	

publication_platforms

Colonne	Type	Contraintes	Référence
publication_id	INTEGER	PK	→ publications (CASCADE)
platform_id	INTEGER	PK	→ platforms (CASCADE)

remunerations

Colonne	Type	Contraintes	Référence
id	INTEGER	PK	
project_service_id	INTEGER	NOT NULL	→ project_services (CASCADE)
user_id	INTEGER	NOT NULL	→ users (RESTRICT)
amount	NUMERIC(12, 2)	NOT NULL	
currency	VARCHAR(3)	NOT NULL	
status	VARCHAR(30)		
created_at	TIMESTAMP WITH TIME ZONE	NOT NULL	

revenues

Colonne	Type	Contraintes	Référence
id	INTEGER	PK	
project_service_id	INTEGER	NOT NULL	→ project_services (CASCADE)
amount	NUMERIC(12, 2)	NOT NULL	
currency	VARCHAR(3)	NOT NULL	
created_at	TIMESTAMP WITH TIME ZONE	NOT NULL	

activity_assignments

Colonne	Type	Contraintes	Référence
activity_id	INTEGER	PK	→ activities (CASCADE)
user_id	INTEGER	PK	→ users (CASCADE)
assignment_role	assignmentrole	NOT NULL	

activity_checklists

Colonne	Type	Contraintes	Référence
id	INTEGER	PK	
activity_id	INTEGER	NOT NULL	→ activities (CASCADE)
label	VARCHAR(255)	NOT NULL	
is_done	BOOLEAN	NOT NULL	
done_by	INTEGER		→ users (SET NULL)
done_at	TIMESTAMP WITH TIME ZONE		
display_order	INTEGER	NOT NULL	

notifications

Colonne	Type	Contraintes	Référence
id	INTEGER	PK	
user_id	INTEGER	NOT NULL	→ users (CASCADE)
type	VARCHAR(50)	NOT NULL	
title	VARCHAR(200)	NOT NULL	
message	TEXT	NOT NULL	
project_id	INTEGER		→ projects (SET NULL)
activity_id	INTEGER		→ activities (SET NULL)
is_read	BOOLEAN	NOT NULL	
created_at	TIMESTAMP WITH TIME ZONE	NOT NULL	

payments

Colonne	Type	Contraintes	Référence
id	INTEGER	PK	
revenue_id	INTEGER	NOT NULL	→ revenues (CASCADE)
amount	NUMERIC(12, 2)	NOT NULL	
paid_at	TIMESTAMP WITH TIME ZONE	NOT NULL	
method	VARCHAR(50)		
notes	TEXT		

Annexe B — Référence de l'API

Générée depuis les routers (51 points d'accès). « Connecté » = tout utilisateur authentifié ; « Public » = aucune authentification. La documentation interactive complète (schémas de requête/réponse) est servie par FastAPI sur /docs.

Méthode	Chemin	Accès	Fonction
POST	/api/auth/login	Public	login
GET	/api/auth/me	Connecté	me
POST	/api/auth/unlock/{user_id}	CEO	unlock_account
GET	/api/users/roles	CEO / DA	list_roles
GET	/api/users/skills	CEO / DA	list_skills
GET	/api/users/team	CEO / DA	team
GET	/api/users	CEO	list_users
POST	/api/users	CEO	create_user
PATCH	/api/users/{user_id}	CEO	update_user
PUT	/api/users/{user_id}/skills	CEO / DA	set_user_skills
GET	/api/clients	CEO / DA	list_clients
GET	/api/clients/{client_id}	CEO / DA	get_client
POST	/api/clients	CEO / DA	create_client
PATCH	/api/clients/{client_id}	CEO / DA	update_client
GET	/api/public/categories	Public	list_categories
GET	/api/public/event-types	Public	list_event_types
GET	/api/public/services	Public	list_services
POST	/api/public/requests	Public	submit_request
GET	/api/requests	CEO / DA	list_requests
GET	/api/requests/{request_id}	CEO / DA	get_request
GET	/api/requests/{request_id}/history	CEO / DA	get_request_history
PATCH	/api/requests/{request_id}/status	CEO / DA	update_request_status
GET	/api/projects	CEO / DA / CM	list_projects
GET	/api/projects/{project_id}	CEO / DA / CM	get_project
POST	/api/projects	CEO / DA	create_project
PATCH	/api/projects/{project_id}	CEO / DA	update_project
GET	/api/activities	CEO / DA / CM	list_activities
GET	/api/activities/{activity_id}	CEO / DA / CM	get_activity
POST	/api/activities	CEO / DA	create_activity
PATCH	/api/activities/{activity_id}	CEO / DA / CM	update_activity
GET	/api/activities/{activity_id}/suggested-users	CEO / DA	suggest_users
POST	/api/activities/{activity_id}/assignments	CEO / DA	assign_user
PATCH	/api/activities/{activity_id}/checklist/{item_id}	CEO / DA / CM	toggle_checklist_item
GET	/api/publications/platforms	CEO / DA / CM	list_platforms
GET	/api/publications	CEO / DA / CM	list_publications
POST	/api/publications	CEO / DA / CM	create_publication
PATCH	/api/publications/{publication_id}	CEO / DA / CM	update_publication

Méthode	Chemin	Accès	Fonction
GET	/api/documents	CEO / DA / CM	list_documents
POST	/api/documents	CEO / DA / CM	upload_document
GET	/api/documents/{document_id}/download	CEO / DA / CM	download_document
DELETE	/api/documents/{document_id}	CEO / DA	delete_document
POST	/api/finance/revenues	CEO	create_revenue
POST	/api/finance/payments	CEO	create_payment
POST	/api/finance/remunerations	CEO	create_remuneration
GET	/api/finance/project-services/{project_service_id}/detail	CEO	get_detail
GET	/api/finance/project-services/{project_service_id}/balance	CEO	get_balance
GET	/api/notifications	Connecté	list_my_notifications
PATCH	/api/notifications/{notification_id}/read	Connecté	mark_as_read
GET	/api/security/events	CEO	list_events
GET	/api/security/locked-accounts	CEO	locked_accounts
WS	/ws?token=...	Connecté (JWT)	websocket_endpoint

Paramètres de requête utiles : /requests?status_filter=, /projects?responsible_id=, /activities?project_id=&mine=, /notifications?unread_only=, /documents?project_id= (obligatoire), /security/events?limit=.

Annexe C — Comment étendre Kimia

Ajouter une prestation avec son formulaire et ses activités (sans code)

- Insérer une ligne dans `services` (rattachée à une catégorie).
- Créer ses champs dans `form_fields` et les lier via `service_form_fields` (ordre, obligatoire ou non). Types gérés par le formulaire public : `text`, `textarea`, `select` (avec options : `{"choices": [...]}`), `date`, `time`, `number`, `email`, `tel`.
- Lier les types d'événements applicables via `service_event_types`.
- Créer ses `activity_templates` (et `checklist_template_items`) : ils seront copiés dans chaque futur projet.

Ajouter une nouvelle ressource API

- Modèle dans le fichier de domaine adapté de `models/` + import dans `models/__init__.py`, puis `alembic revision --autogenerate`.
- Schemas `XCreate` / `XUpdate` / `XRead` dans `schemas/`.
- Router dans `routers/`, protégé par `require_roles(...)`, branché dans `main.py`. Après commit : `manager.broadcast_sync({"type": "x.created", "data": {"id": ...}})`.
- Logique multi-modèles ? La mettre dans `services/`, pas dans le router.

Ajouter un écran à l'application interne

- Méthodes d'API dans `services/api.js`.
- Module dans `assets/js/pages/` respectant le contrat de la section 4.3.
- Une page HTML de 10 lignes qui appelle `boot(page)` (copier une page existante) et une entrée dans `MENU` de `components/shell.js`.
- Si la page doit réagir à un nouvel événement, l'ajouter à sa liste `events`.

Volumétrie : backend 3037 lignes Python (38 fichiers) + 229 lignes de tests ; frontend 838 lignes JS, 621 lignes HTML, 134 lignes CSS.